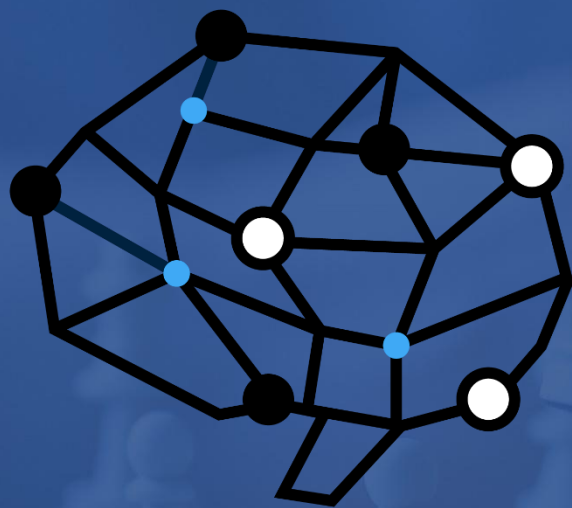




-黑白棋程式實作-

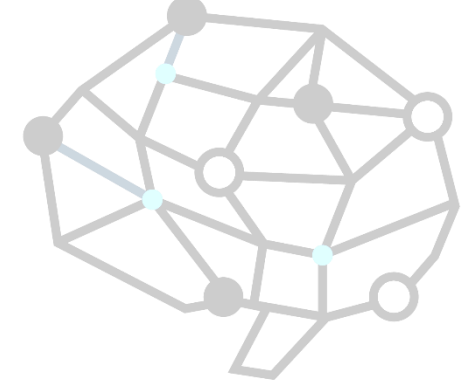
周信宏

套件規劃

- othello
 - OthelloUtil.py : 黑白棋運作邏輯
 - OthelloGame.py : 控制黑白棋遊戲進行
- bots
 - Random.py : 自動隨機下



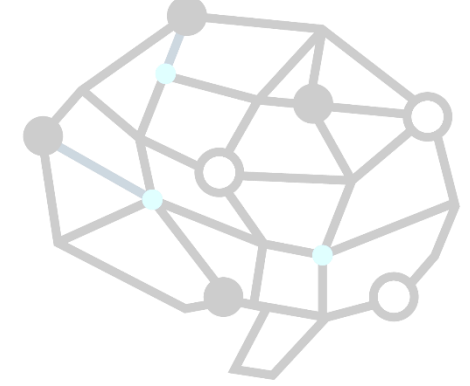
OthelloUtil.py



方法

- `getValidMoves(board: np.array, color: int) - Nx2 array`
取得盤面上某方的合法位置有哪些
- `isValidMove(board: np.array, color: int, position: tuple) - boolean`
檢查落子是否合法
- `executeMove(board: np.array, color: int, position: tuple) - void`
執行落子

OthelloUtil - getValidMoves



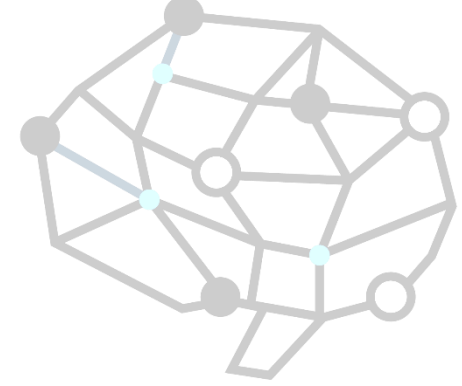
```
import numpy as np

def getValidMoves(board, color):
    moves = set()
    for y,x in zip(*np.where(board==color)):
        for direction in [(1,1),(1,0),(1,-1),(0,-1),(-1,-1),(-1,0),(-1,1),(0,1)]:
            flips = []
            HIDDEN CODE, HIDDEN CODE
    return np.array(list(moves))
```

從每個等於color的位置向8方延伸出去
(HIDDEN CODE在下一頁)

OthelloUtil - getValidMoves

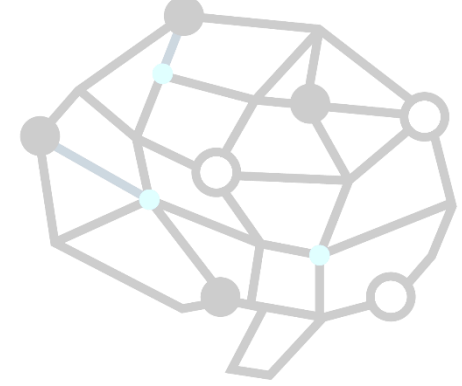
```
for size in range(1, len(board)):
    ydir = y + direction[1] * size
    xdir = x + direction[0] * size
    if xdir >= 0 and xdir < len(board) and ydir >= 0 and ydir < len(board):
        if board[ydir][xdir]==-color:
            flips.append((ydir, xdir))
        elif board[ydir][xdir]==0:
            if len(flips)!=0:
                moves.add((ydir, xdir))
            break
        else:
            break
    else:
        break
```



如果延伸過程只有對手棋子
且已經延伸到空位
就代表該空位是合法步

OthelloUtil - isValidMove

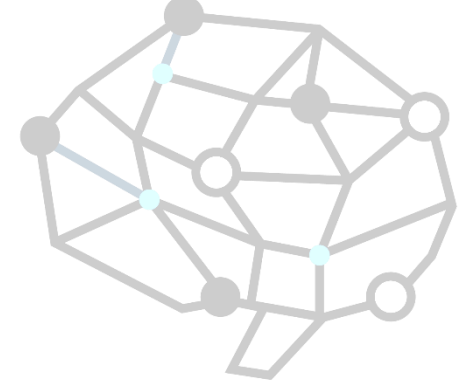
```
def isValidMove(board, color, position):  
    valids=getValidMoves(board, color)  
    if len(valids)!=0 and (valids==np.array(position)).all(1).sum()!=0:  
        return True  
    else:  
        return False
```



落子位置存在於getValidMoves中
則回傳True

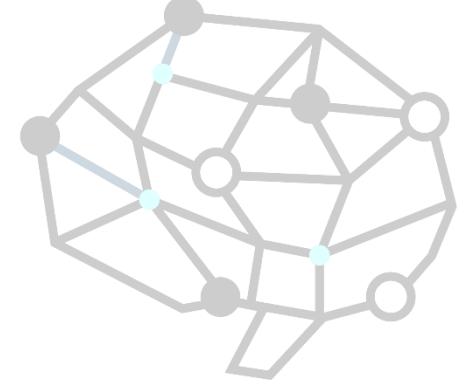
OthelloUtil - executeMove

```
def executeMove(board, color, position):  
    y, x = position  
    board[y][x] = color  
    for direction in [(1,1),(1,0),(1,-1),(0,-1),(-1,-1),(-1,0),(-1,1),(0,1)]:  
        flips = []  
        valid_route=False  
        HIDDEN CODE, HIDDEN CODE  
        if valid_route:  
            board[tuple(zip(*flips))]=color
```



先將落子位置改成自己棋子
接下來和getValidMoves一樣
掃描落子位置的8個方向

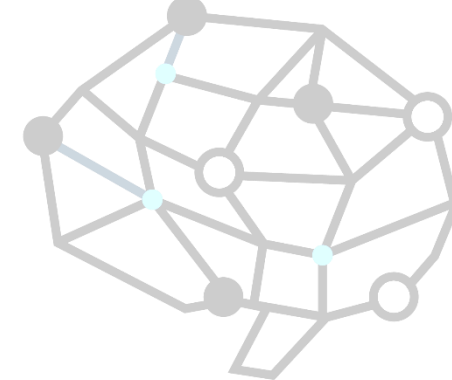
OthelloUtil - executeMove



```
for size in range(1, len(board)):
    ydir = y + direction[1] * size
    xdir = x + direction[0] * size
    if xdir >= 0 and xdir < len(board) and ydir >= 0 and ydir < len(board):
        if board[ydir][xdir]==-color:
            flips.append((ydir, xdir))
        elif board[ydir][xdir]==color:
            if len(flips)>0:
                valid_route=True
            break
        else:
            break
    else:
        break
```

延伸過程記錄對手棋子座標
如果遇到自己棋子
則將記錄中的位置翻棋
如果遇到空位則清空記錄
代表該路徑無法翻棋

OthelloGame



屬性

➤ `n: int` (棋盤大小)

➤ `current_player: int`

OthelloGame.BLACK=1
OthelloGame.WHITE=-1

方法

➤ `move(position: tuple): void`

➤ `isEndGame(): result`

➤ `play(): void`

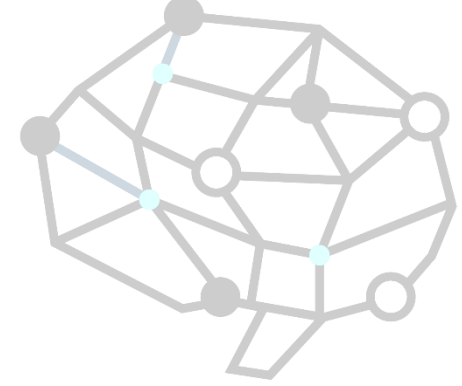
➤ `showBoard(): void`

未結束: None
黑贏: 1
白贏: -1
平手: 0

OthelloGame

```
import numpy as np
class OthelloGame(np.ndarray):
    BLACK = 1
    WHITE = -1
    def __new__(cls, n):
        return super().__new__(cls, shape=(n,n), dtype='int')

    def __init__(self, n):
        self.n=n
        self.current_player=OthelloGame.BLACK
        self[np.where(self!=0)]=0
        self[int(n/2)][int(n/2)]=OthelloGame.WHITE
        self[int(n/2)-1][int(n/2)-1]=OthelloGame.WHITE
        self[int(n/2)-1][int(n/2)]=OthelloGame.BLACK
        self[int(n/2)][int(n/2)-1]=OthelloGame.BLACK
```

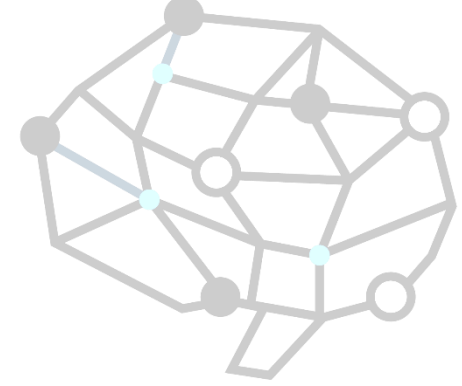


OthelloGame繼承numpy陣列
將黑子定義為1、白子定義為-1、空的為0

設定先手為黑子
並初始化棋盤

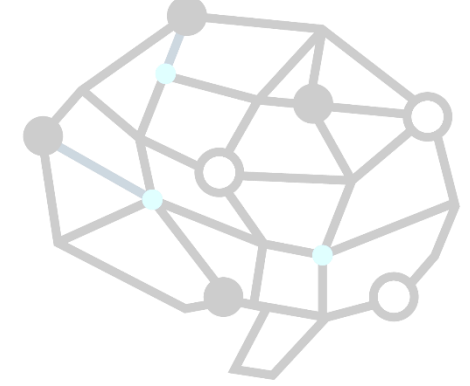
OthelloGame

```
import numpy as np
from othello.OthelloUtil import getValidMoves, executeMove, isValidMove
class OthelloGame(np.ndarray):
    ...
    ...
    ...
    def move(self, position):
        if isValidMove(self, self.current_player, position):
            executeMove(self, self.current_player, position)
            self.current_player=-self.current_player
        else:
            raise Exception('invalid move')
```



如果該步合法則執行翻棋
並將current_player改成另一位玩家

OthelloGame

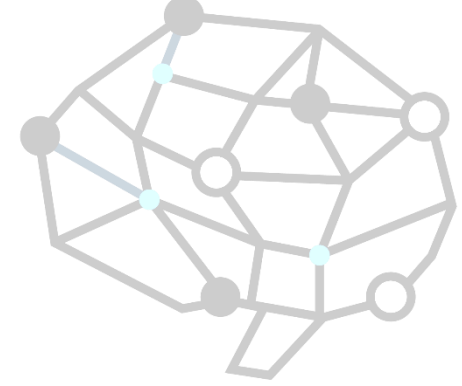


```
class OthelloGame(np.ndarray):  
    ...  
    def isEndGame(self):  
        white_valid_moves=len(getValidMoves(self, OthelloGame.WHITE))  
        black_valid_moves=len(getValidMoves(self, OthelloGame.BLACK))  
        if white_valid_moves==0 and black_valid_moves==0:  
            v,c=np.unique(self, return_counts=True)  
            white_count=c[np.where(v==OthelloGame.WHITE)]  
            black_count=c[np.where(v==OthelloGame.BLACK)]  
            if white_count>black_count:  
                return OthelloGame.WHITE  
            elif black_count>white_count:  
                return OthelloGame.BLACK  
            else:  
                return 0  
        else:  
            return None
```

如果兩方都無合法步可走
代表遊戲結束
計算雙方棋數
回傳勝方，平手則回傳0
尚未結束則回傳None

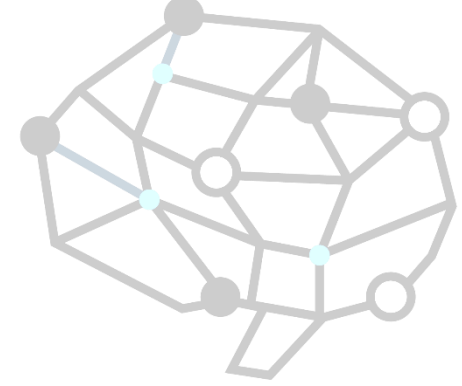
OthelloGame

```
class OthelloGame(np.ndarray):
    ...
    def showBoard(self):
        corner_offset_format='{:^'+str(len(str(self.n))+1)+'}'
        print(corner_offset_format.format(""), end="")
        for i in range(self.n):
            print('{:^3}'.format( chr(ord('A')+i) ), end="")
            print()
        print(corner_offset_format.format(""), end="")
        for i in range(self.n):
            print('{:^3}'.format('-'), end="")
            print()
        for i in range(self.n):
            print(corner_offset_format.format(i+1), end="")
            for j in range(self.n):
                if isValidMove(self, self.current_player, (i,j)):
                    print('{:^3}'.format('■'), end="")
                else:
                    print('{:^3}'.format(self[i][j]), end="")
            print()
```



顯示棋盤

OthelloGame



```
class OthelloGame(np.ndarray):
    ...
    def play(self, black, white):
        while self.isEndGame() == None:
            print('{: ^30}'.format( ' Player '+str(self.current_player)+' ' ))
            self.showBoard() # 顯示棋盤
            if len(getValidMoves(self, self.current_player))==0: # 無合法步
                self.current_player=-self.current_player
                continue
            if self.current_player==OthelloGame.WHITE:
                position=white.getAction(self.clone(), self.current_player)
            else:
                position=black.getAction(self.clone(), self.current_player)
            try:
                self.move(position)
            except:
                print('invalid move', end='\n\n')
                continue
```

black及white為物件
需要有回傳位置的函數getAction
OthelloGame會將棋盤及當前玩家
傳入getAction

執行直到isEndGame() != None
過程如果當前玩家無合法步
則直接跳過

執行方法

```
from othello import OthelloGame

game=OthelloGame(n=8)

class Human:
    def getAction(self, game, color):
        print('input coordinate:', end='')
        coor=input()
        return (int(coor[1])-1, ord(coor[0])-ord('A'))

game.play(black=Human(), white=Human())
```



即可在console看到文字介面的棋盤
■代表合法步
輸入座標即可讓getAction方法
回傳位置給OthelloGame

```
##### Player 1 #####
  A  B  C  D  E  F  G  H
-  -  -  -  -  -  -  -
1  0  0  0  0  0  0  0
2  0  0  0  0  █  █  0  0
3  0  0  0  0  -1  0  -1  0
4  0  0  1  1  1  -1  █  0
5  0  0  0  1  -1  █  0  0
6  0  0  0  0  █  █  0  0
7  0  0  0  0  0  0  0  0
8  0  0  0  0  0  0  0  0
input coordinate:
F5|
```

執行方法

```
from othello import OthelloGame
from othello.bots.Random import BOT

game=OthelloGame(n=8)

class Human:
    def getAction(self, game, color):
        print('input coordinate:', end='')
        coor=input()
        return (int(coor[1])-1, ord(coor[0])-ord('A'))

game.play(black=Human(), white=BOT())
```



使用othello.bots.Random.BOT

執行方法

```
import numpy as np
from othello.OthelloUtil import getValidMoves

class BOT():
    def __init__(self, *args, **kwargs):
        pass

    def getAction(self, game, color):
        valids=getValidMoves(game, color)
        position=np.random.choice(range(len(valids)), size=1)[0]
        position=valids[position]
        return position
```



othello.bots.Random.BOT內容
隨機取一個合法步

A blue-tinted background image showing a hand moving a chess piece on a chessboard. The text "THE END." is centered in the middle of the image, underlined.

THE END.
